



Team members:

Erickson Mijangos

Israel Mejia

Dave Contreras

EE471: Design of Control Systems

Fall 2020

Final Project: Adaptive Cruise Control

12/12/2020

Abstract.

This lab will be going over simulating Adaptive Cruise Control on matlab. In order to make our cruise control be as smooth as possible we will be developing a different array of controllers for this system and we will figure out which controller gives us the best possible output response for this situation. The Adaptive Cruise Control consists of two cars one being the ego car and the other being the lead car. Sensors are applied to the ego car that will be able to sense and calculate the relative distance between both cars. This information will be used to have our ego car follow the lead car at the same velocity while keeping a safe distance. The Ego car can be set to higher speed but will slow down as it approaches the lead car and will keep the proper speed in order to accommodate a safe distance between both cars.

Percentage of efforts for writing:

- Dave Contreras for lab report = 33%
- Israel Mejia for lab report = 33%
- Erickson Mijangos for lab report = 34%

Percentage of efforts for project:

- Dave Contreras for GUI development, PP presentation and supporting documentation = 34%
- Israel Mejia for simulation model and system modeling = 33%
- Erickson Mijangos for simulation model and system modeling = 33%

Table of Content:

- Introduction - Pages 3-4
- Objective - Page 4
- Problem Description - Pages 4-5
- Simulink Modeling - Pages 5-11
- Conclusion - Page 12
- Future Development - Page 12
- References - Page 12

Introduction:

Since the modern world most systems are automated, the project intended to develop an Adaptive Cruise Controller. The Adaptive Cruise Controller is a system that allows the vehicle to adapt the speed according to the surroundings. A radar system system is used to detect whether the vehicle is safe enough to travel at the speed that is desired or decelerate to prevent the collision of any leading obstacle. In simple words if the radar system senses that it detecting a slow moving vehicle is ahead the sensor will calculate to slow down to prevent collision. The sensor uses the ego vehicle's velocity to calculate whether the distance between the lead and ego are safe enough or it has to slow when it surpasses the safe referenced distance it calculated. When the ego vehicle detects that it is further from the lead car and does not detect that it has surpassed a referenced safe distance, the ACC can take the opportunity to speed up to its desired set speed. In Figure 1 shows a simple drawing of both cases where the cruise controller is further from the safe distance (top), and if it has surpassed the safe distance (bottom). A simple state flow of the operation of the ACC is represented in a state machine representation in Figure 2.

The following rules are used to determine the ACC system operating mode:

- If $D_{rel} \geq D_{safe}$, then speed control mode is active. The control goal is to track the driver-set velocity, V_{set} .
- If $D_{rel} < D_{safe}$, then spacing control mode is active. The control goal is to maintain the safe distance, D_{safe} .

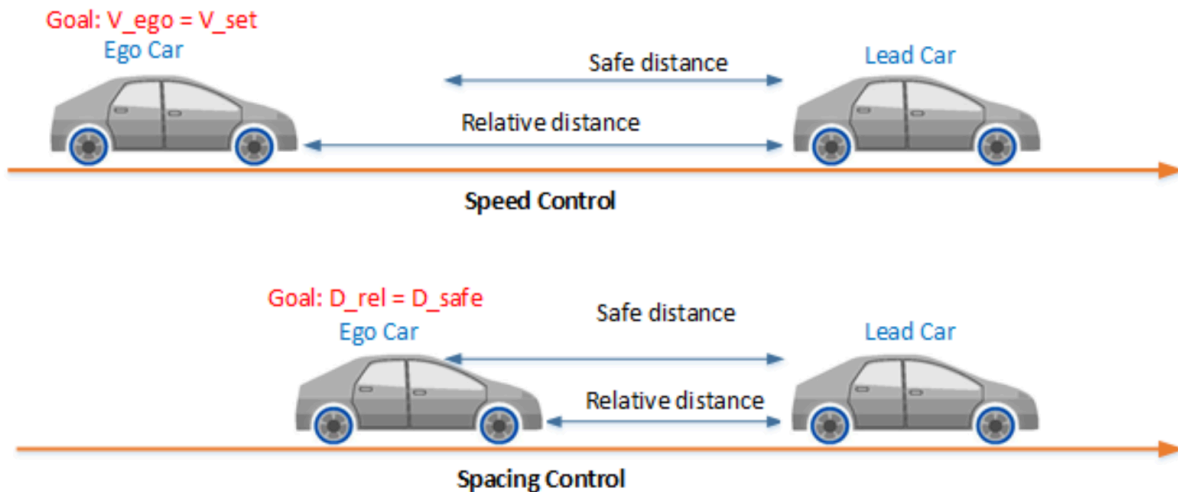


Figure 1: Speed Control and Space Control.

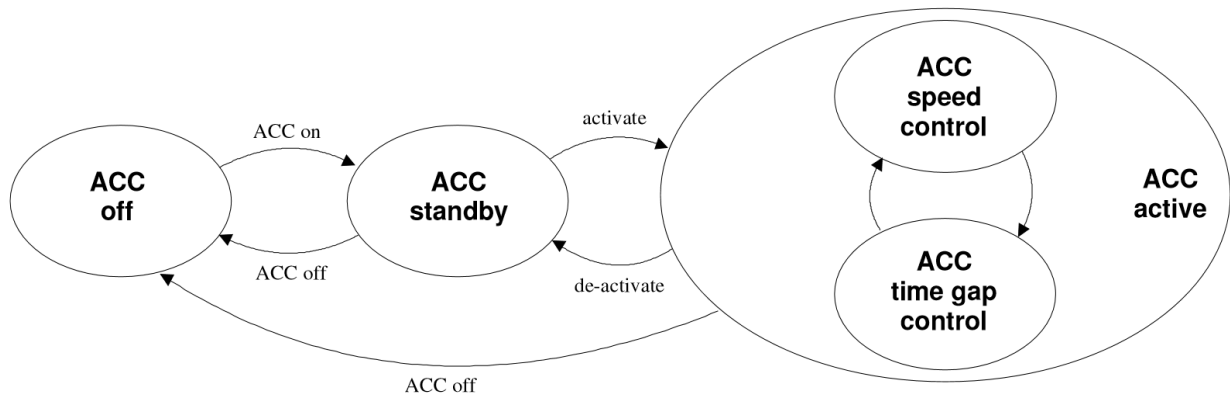


Figure 2: State Machine Representation of how the ACC operates when using the safe distance as a marker.

Objective:

With the quick overview of some parameters involved for the radar system of the ego vehicle, we can use those parameters to design a control system that is able to develop an acceleration signal to determine whether the ego should speed up or slow down, once it detects that it is a safe distance marker. The reason acceleration as a source to control the vehicle plants since acceleration itself involves the calculation in the rate that velocity can change. Therefore, it is a reliable source of information for the vehicle plant to use for any changes it needs to make to meet system requirements.

Since acceleration is a reliable signal to create a simulating movement of both vehicles, it can also be used to develop a control system based purely on acceleration and few initial conditions of both vehicles, such as movements and the speed it is traveling while accelerating within those speeds. Referencing from Figure 1, the system requirements, such as movement behavior of the vehicle in 1D (position and velocity of vehicle over time), set speed desired and safe distance to reference, set by the user. The development of the control will drastically involve creating a sensor system that uses both lead and ego vehicle position and velocity signal in order to make comparison with the desired set speed and safe distance to create an acceleration signal which a controller can use to meet the system requirements.

Problems Description:

Although the objectives have covered some emphasis to develop the sensor and controller for the ego vehicle, one of the main aspects of the project is to develop an acceleration signal that can dictate how the ego vehicle should respond according to the lead vehicle behavior. Since it involves multiple signals, it is used as an attempt to linearize the sensors to develop an acceleration signal for the ego vehicle. After developing a control system for both the ego vehicles sensor and controller, it is desirable to have a test simulation to see how effective the response of the sensor and controller is for the ego vehicle to prevent collision. Although MATLAB has multiple tool boxes to develop the simulation, another constraint other

than designing an effective controller is the length of simulation space created in MATLAB to simulate both vehicles moving.

Simulink Modeling:

With all the available parameters that can be used to model the ACC, Simulink is used extensively for the project to both model and simulate an animation of both vehicle's movement. To provide simplicity, and make things a little more user-friendly simulation, a GUI is developed in order to enter the desired parameters and run the simulation using the automated driving toolbox from MATLAB. Another package used from MATLAB is GUI app developer which can call the desired import and export variable used on the model to simulate the control systems and animation of the cars themselves. In Figure 3 shows the overall Simulink of the type of test signals that is manual set, a subsystem of the ACC developed, exporting variables to the GUI and blocks such as scene, sensor, and vehicles in their respective location to simulate the animation.

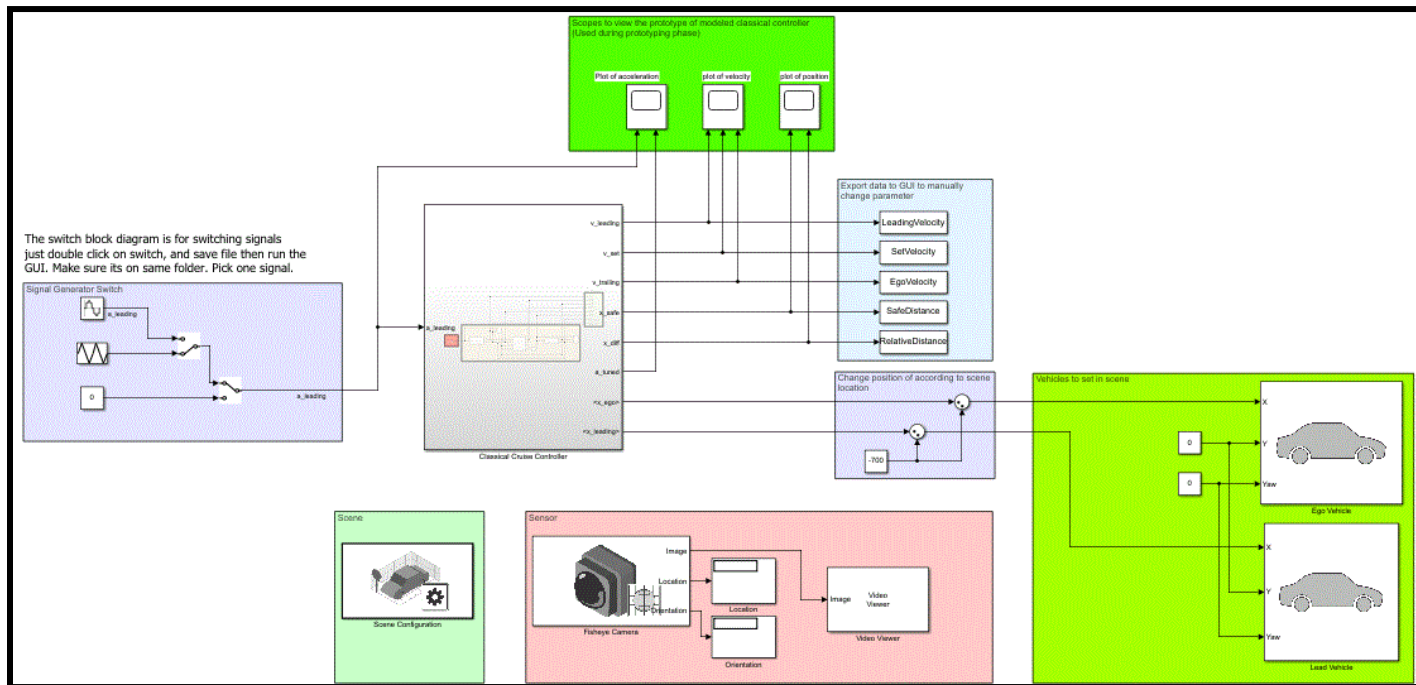


Figure 3: Overall Model with all subsystems of the input signals, ACC, and other blocks used to simulate the animation and export data to plot in GUI.

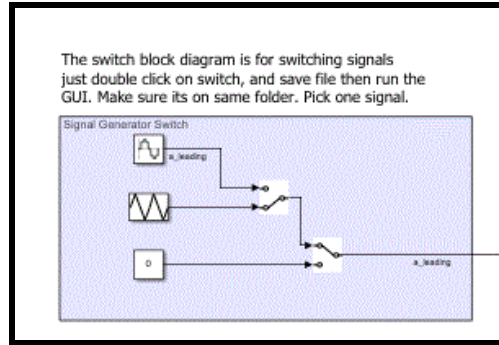


Figure 4: Input

Our overall Simulink model gives us the ability to choose the type of input acceleration signal given to our lead car. From the input, we can choose from an option of a sinusoidal, triangular and no acceleration (constant velocity). These inputs are fed into our main subsystem, which outputs the position values into our scene for 3D animation. Our main subsystem also outputs the relative distance and velocity of the lead and ego car to be graphed in our GUI.

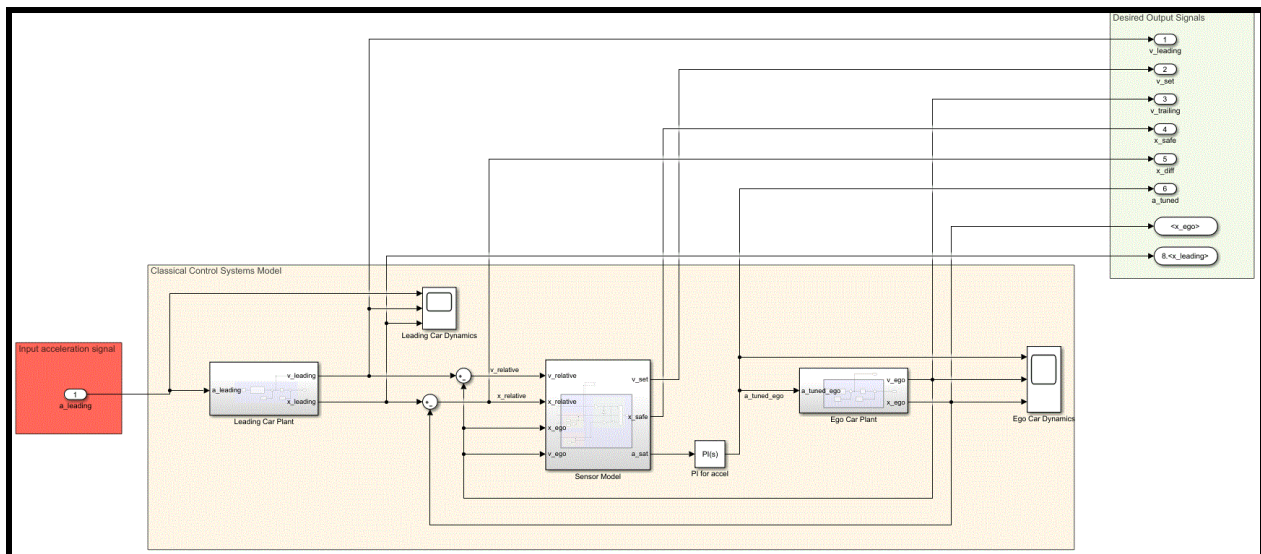


Figure 5: Main Subsystem

Our main subsystem consists of 3 subsystems. The three subsystems are the ego car subsystem, lead car subsystem and the Adaptive Cruise Control subsystem. The outputs are fed into our 3D simulations model and GUI to animate and plot the response of our system. This is all controlled by our GUI, that gives us control of the system to simulate different scenarios.

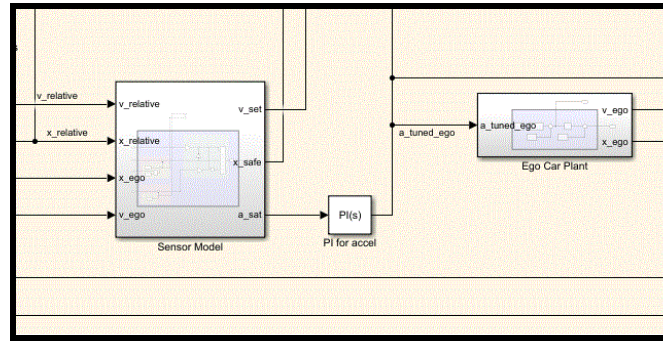


Figure 6: PI Block

Our main subsystem also has our sensor model output for acceleration fed into the PI controller block that is then fed into the ego car subsystem. For the system, we found out that it was only necessary to implement a PI controller. If we included a full PID controller, it would make our system response lag, resulting in the ego car response to be slow and less accurate.

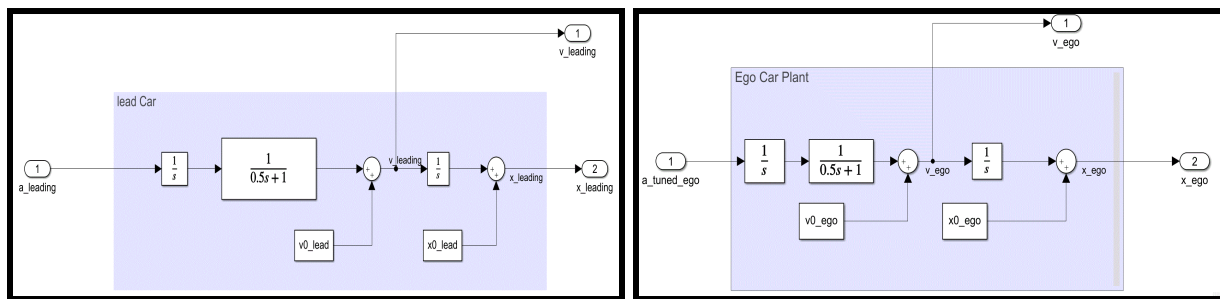


Figure 7 and 8: Lead and Ego Car Subsystem

$$\frac{1}{0.5s + 1}$$

Figure 9: Car Acceleration Transfer Function

Both our lead car and ego car are controlled by a simple transfer function that controls the vehicle acceleration dynamics. The dynamics are put through integrators in order to get our vehicles velocity and position value. The initial velocity and position of the cars can be changed through the right after each designated integrator block.

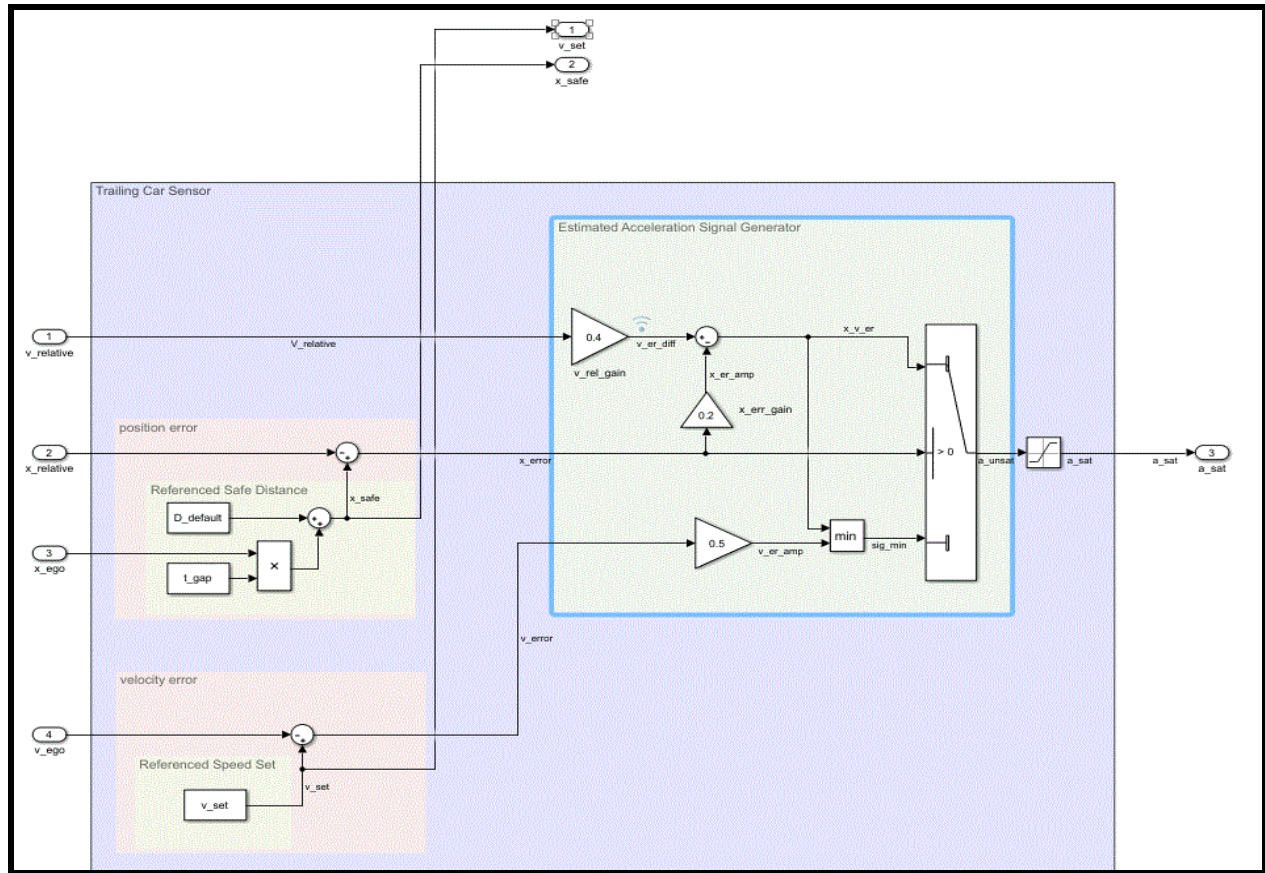


Figure 10: Ego Sensor Model

Our ego car sensor basically takes in relative velocity and distance and decides if the ego should go at the set speed or if it should try to match the leading car's velocity depending on the relative distance between the two cars. As the ego car approaches the leading car our switch block will change over to from the set velocity signal and instead pass through a signal such that the ego car can match the lead car's velocity while keeping a safe distance between the two cars.

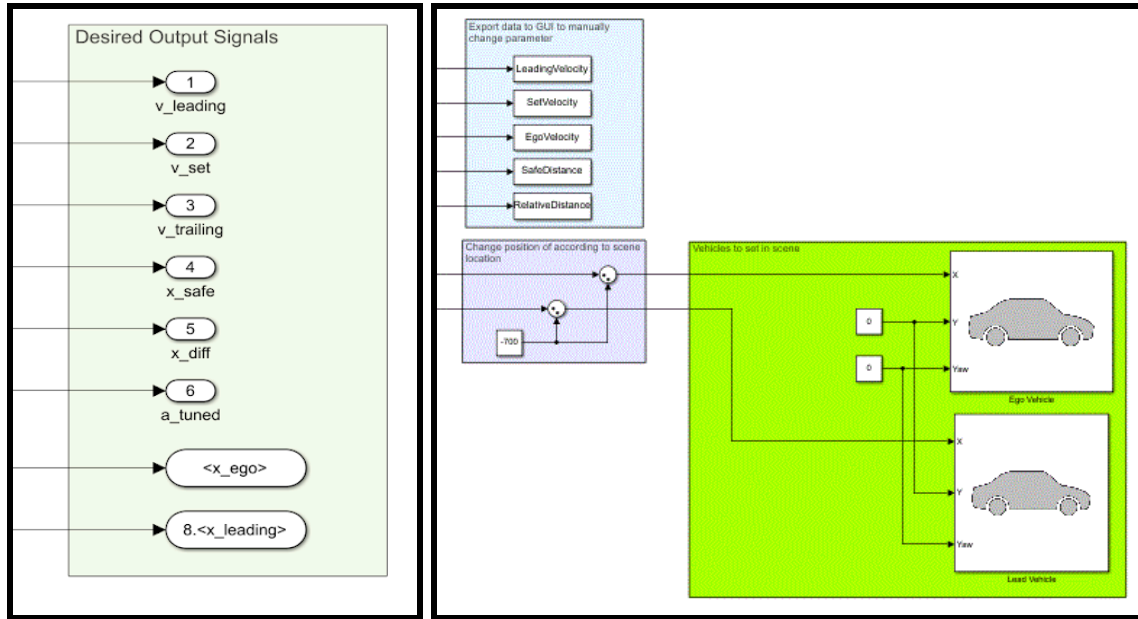


Figure 11 and 12: Desired Outputs and Designated Response Inputs

These are desired outputs that come out of the main subsystem model; these outputs are fed into our GUI for export and Vehicles block to be animated and graphed into our GUI.

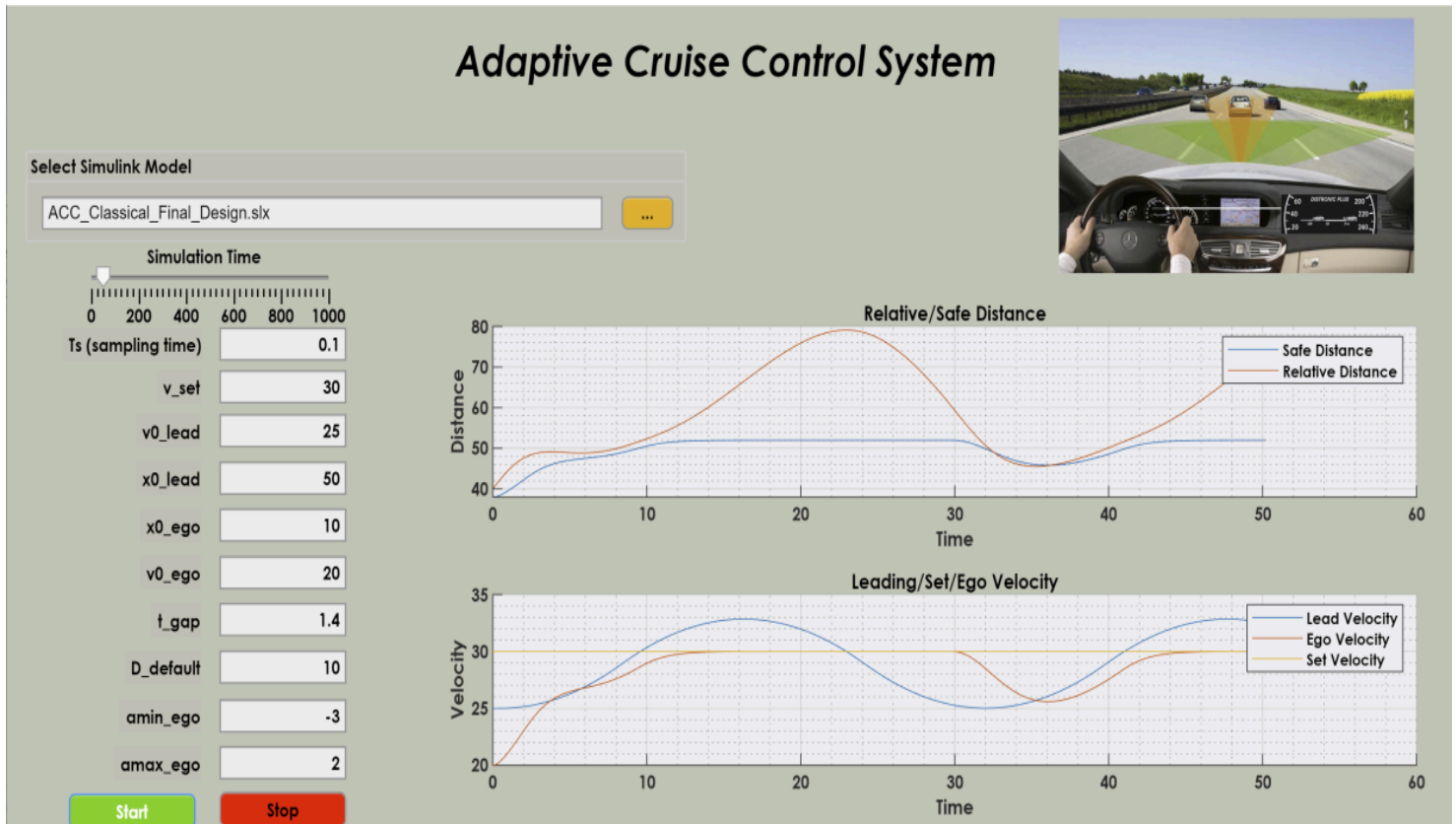


Figure 13. Adaptive Cruise Control System GUI

This is the Adaptive Cruise Control GUI that makes adjusting our system parameters more user friendly. The GUI also gives a plot for the safe distance and relative distance between the two cars as well as a plot for the velocity of both cars and indicates what the set velocity is set at for the ego car. The GUI gives the ability to start and end the simulation at any moment while also being able to control how long the user wants to run the simulation for. From the graphs above it can be seen that the ego car will not try to go past the safe distance while trying to match the lead car's velocity and capping its velocity at the set velocity. Whenever the ego car needs to catch up to the lead car it sticks to the set velocity of 30m/s until it reaches a safe distance behind the lead car and then starts to match the velocity of the lead car.



Figure 14 and 15: 3D Car Animation

The 3D car animation takes in the x values for our ego car (red car) and lead car (green car) and plots them on the designated scene. The scene we have chosen to demonstrate our cruise control is a straight road scene. This animation shows how the ego cars response responds as it approaches the lead car and demonstrates that our Adaptive Cruise Control is working properly. A better look at how the simulation works can be seen in the Adaptive Cruise Control group video.

Conclusion.

Our Adaptive Cruise Control was very responsive and was able to give us our ego car a good response in order to properly adjust its velocity as it approached the car ahead of it. We were able to make a responsive controller that was able to quickly shift from a regular cruise control to the Adaptive Cruise Control once it came in a close proximity of a car ahead of it while also keeping a safe distance. This type of cruise controller is very useful for cars driving on the highway and for long distance driving.

Future development:

Although the simulink has successfully animated the vehicles and the ACC implemented in action. The model can also be used to develop on 1D dimensional track where we can experiment the results according to the model from simulink. Although the acceleration signal was approximate, a state space model can be used to design a controller and sensor much more effectively compared to the classical model that is developed.

References:

- <https://www.mathworks.com/videos/getting-started-with-the-vehicle-dynamics-blockset-vdbs--1555080044769.html>
- <https://www.mathworks.com/products/vehicle-dynamics.html>
- <https://www.mathworks.com/products/3d-animation.html>
- <https://www.mathworks.com/help/mpc/ug/adaptive-cruise-control-using-model-predictive-controller.html>
- https://www.mathworks.com/help/matlab/creating_guis/create-a-simple-app-or-gui-using-app-designer.html
- <https://www.mathworks.com/help/matlab/app-designer.html>